PROYECTO BACKEND / GUÍA TÉCNICA

Users & Studio Ghibli API

Python · Flask · MySQL
Docker · JWT · GitHub Actions · AWS EC2

Del arranque local al despliegue y la recuperación

Código, decisiones de arquitectura, permisos por rol, pruebas y procedimiento operativo para una instancia EC2 con HTTPS.

Incluido: CRUD completo, autenticación, autorización, integración Ghibli, documentación Swagger, migraciones, pipeline y scripts de operación.

Alcance: despliegue de una sola EC2. Requiere preparar los recursos de tu cuenta; no se ha desplegado en AWS durante la elaboración del proyecto.

Preparado para Miguel Ángel
26 de octubre de 2026

Cómo utilizar esta guía

El código del ZIP es la fuente ejecutable. Copia los comandos largos desde los archivos Markdown originales para conservar comillas y saltos de línea. En el PDF pueden dividirse visualmente por espacio.

01 / Inicio y contrato REST

Instalación local, Swagger, endpoints y ejecución de pruebas.

02 / Arquitectura y decisiones

Responsabilidades, autenticación, transacciones, fallos y escalabilidad.

03 / AWS EC2 paso a paso

IAM, Docker, GitHub, HTTPS, secretos, primer despliegue y validación.

04 / Evidencia de validación

Qué se ejecutó, qué pasó y qué requiere infraestructura adicional.

05 / Fuentes

Documentación técnica consultada para las herramientas utilizadas.

Orden recomendado: ejecutar en local, revisar la matriz de roles, correr las pruebas, preparar un entorno de staging y finalmente configurar production.

Users & Studio Ghibli API

Proyecto Python 3.12 + Flask, MySQL 8.4, SQLAlchemy, JWT, Redis, Swagger y Docker. Incluye CI/CD para AWS EC2 mediante GitHub Actions, ECR y Systems Manager.

Inicio rápido (Windows, macOS o Linux)

Instala Docker Engine con Compose v2 o Docker Desktop y Python 3.12. En Windows utiliza contenedores Linux; los comandos siguientes pueden ejecutarse en PowerShell (usa `python` o `py -3.12`). Descomprime el ZIP y abre una terminal dentro de `ghibli-users-api`.

```
python scripts/init_env.py
docker compose up --build -d
docker compose ps -a
docker compose logs migrate api
docker compose exec api flask --app wsgi:app create-admin
```

El script genera contraseñas aleatorias; no existe usuario ni contraseña por defecto. La CLI solicita nombre, correo y contraseña (mínimo 12 caracteres; no se muestra). `.env` nunca se sube a Git. Si ya existe, el generador se detiene para evitar reemplazar secretos.

MySQL y Redis arrancan primero; un contenedor separado ejecuta las migraciones y termina con código 0. Después inicia Gunicorn. El estado `Exited (0)` de `migrate` es normal.

- Swagger UI: http://localhost:8000/docs
- OpenAPI JSON: http://localhost:8000/openapi.json
- Readiness: http://localhost:8000/health/ready
- Liveness: http://localhost:8000/health/live

En Swagger, ejecuta `POST /api/v1/auth/login` con el admin creado. Copia `access_token`, pulsa **Authorize** y pega solamente el token (sin escribir `Bearer`). Crea usuarios y prueba los roles.

Ejemplo de uso

Cuerpo de login:

```
{"email":"admin@example.com","password":"TU_PASSWORD_DE_12_O_MAS_CARACTERES"}
```

Para crear un usuario, utiliza `POST /api/v1/users` y el encabezado `Authorization: Bearer <token>`:

```
{
  "name": "Ana",
  "email": "ana@example.com",
  "password": "CAMBIA_ESTA_PASSWORD_2026",
  "role": "films"
}
```

Inicia sesión con Ana y llama a `GET /api/v1/ghibli/films`. `GET /api/v1/ghibli/people` devuelve 403. El admin puede consultar las cinco colecciones. No se permite registro público ni asignación de roles por usuarios normales.

Contrato REST

Método y ruta	Permiso	Resultado
POST /api/v1/auth/login	Público, limitado	JWT de 15 minutos
POST /api/v1/auth/logout	Autenticado	Revoca el token actual, 204
GET /api/v1/users/me	Autenticado	Perfil propio
POST /api/v1/users	admin	Creación, 201
GET /api/v1/users?page=1&per_page=20	admin	Lista paginada, máximo 100
GET /api/v1/users/{id}	admin	Un usuario
PATCH /api/v1/users/{id}	admin	Actualización parcial
DELETE /api/v1/users/{id}	admin	Borrado físico, 204
POST /api/v1/users/bulk-delete	admin	Borrado atómico de IDs
GET /api/v1/ghibli/{resource}?limit=50	admin o rol igual al recurso	Colección
GET /api/v1/ghibli/{resource}/{uuid}	admin o rol igual al recurso	Detalle

Borrado múltiple: `{"ids":[2,3,4]}`; máximo 100, IDs repetidos se deduplican. Si uno no existe o se eliminaría al último admin, toda la operación se revierte. Se usa POST para la operación masiva para evitar problemas de interoperabilidad con cuerpos en DELETE. La actualización usa PATCH, no reemplazo completo PUT.

Errores: 400 JSON mal formado; 401 credenciales/token; 403 rol; 404 inexistente; 409 correo duplicado o último admin; 413 cuerpo grande; 422 validación; 429 cuota; 502 proveedor inválido; 503 MySQL/Redis; 504 timeout Ghibli. Las contraseñas y hashes nunca forman parte de las respuestas.

Ejecutar pruebas y revisión

```
python -m venv .venv
# Linux/macOS:
source .venv/bin/activate
# Windows PowerShell, en lugar del comando anterior:
# .\.venv\Scripts\Activate.ps1
pip install -r requirements-dev.txt
ruff check .
pytest
pip-audit -r requirements.txt
```

Sin variables adicionales, las pruebas utilizan SQLite temporal, fakeredis y HTTP simulado. No acceden al Ghibli externo. GitHub Actions ejecuta además las pruebas con MySQL 8.4 real y Redis real; comprueba bloqueo concurrente del último administrador y límites de login. No uses una base con datos reales para las pruebas: limpian la tabla users.

Para ejecutarlas contra servicios de prueba propios:

```
export TEST_DATABASE_URL='mysql+pymysql://tester:password@127.0.0.1:3306/ghibli_test'
export TEST_REDIS_URL='redis://127.0.0.1:6379/15'
pytest
```

El pipeline exige mínimo 80% de cobertura y bloquea la publicación si fallan pruebas, lint o auditoría de dependencias. Dependencias Python directas y transitivas quedan fijadas a las versiones instaladas. Dependabot propone actualizaciones; deben volver a probarse.

Estructura

- `app/__init__.py`: factory, configuración, errores, logging, CLI.
- `app/routes.py`: HTTP, documentación y autorización.
- `app/schemas.py`: validación y serialización sin campos secretos.
- `app/services.py`: transacciones, invariantes y operaciones de usuarios.
- `app/security.py`: JWT, revocación y consulta de permisos actuales.
- `app/models.py`: entidades y restricciones de base de datos.
- `app/ghibli.py`: cliente externo, caché y errores del proveedor.
- `migrations/`: historial Alembic; no se usa create_all en producción.
- `tests/`: pruebas de contrato, seguridad, errores e integración.
- `deploy/`: Compose productivo, HTTPS, despliegue, backup y políticas IAM.
- `.github/workflows/ci-cd.yml`: pruebas, construcción y despliegue.
- `docs/`: decisiones, operación AWS, fuentes y validación realizada.

Seguridad y operación

JWT firmado HS256, emisor y audiencia verificados, expiración de 15 minutos y contraseña con scrypt. Un cambio de contraseña o rol incrementa token_version e invalida sesiones anteriores. La baja invalida el acceso porque se consulta el usuario en cada petición. Logout guarda el jti en Redis hasta que vence. Si Redis falla, el acceso falla cerrado. Se requiere nuevo login al vencer; no hay refresh token.

Se limita login por IP (5/minuto) y peticiones generales por IP (120/minuto). Es un control inicial, no reemplaza protección perimetral ni MFA. CORS no se abre globalmente. No se usan cookies de autenticación. HTTPS es obligatorio fuera de localhost. `TRUST_PROXY=true` solo cuando Gunicorn está detrás del proxy definido y su puerto no es público.

Las lecturas de Ghibli tienen caché compartida de 60 segundos, timeout de conexión/lectura y un reintento acotado de GET. El recurso es una lista permitida y el ID debe ser UUID; no se recibe una URL arbitraria del cliente. Ghibli es una API comunitaria externa sin SLA contratado.

El Compose local publica 8000 solamente en localhost; MySQL/Redis no publican puertos. Para AWS usa exclusivamente el Compose productivo y los pasos de `docs/AWS_EC2.md`. No ejecutes `flask run` en producción.

Actualizaciones y datos

```
docker compose down
# Volver a iniciar sin borrar datos:
docker compose up --build -d
```

Los volúmenes conservan MySQL/Redis. `docker compose down -v` BORRA los datos: no usar en producción. Cambiar MYSQL_PASSWORD en `.env` no cambia una contraseña que ya existe en MySQL: debe rotarse con SQL y actualizar la configuración de manera coordinada.

El proyecto entrega una base de despliegue en una EC2, no alta disponibilidad. Lee `docs/ARQUITECTURA.md` para escalar con ALB, múltiples instancias, RDS y Redis compartido. Los recursos de AWS, DNS, roles IAM y variables de GitHub deben crearse en tu cuenta. No se ha ejecutado un despliegue en AWS como parte de este entregable.

Imágenes y S3

El alcance no requiere subir imágenes: los datos Ghibli se devuelven en JSON, incluidas sus URLs externas. No se implementa un endpoint ficticio de subida. Se incluye S3 para backups opcionales y una propuesta concreta para futuros avatares privados en `docs/AWS_EC2.md`.

Arquitectura y decisiones técnicas

1. Flujo de una petición

Cliente HTTPS -> Caddy -> Gunicorn -> Flask -> schema -> autorización -> servicio -> SQLAlchemy -> MySQL. El cliente Ghibli usa HTTPS hacia un proveedor con URL configurable. Redis mantiene revocaciones, cuotas y caché compartida entre los workers.

Flask se crea mediante application factory. Las extensiones se inicializan sin acoplarse a una instancia global. Las vistas gestionan el contrato HTTP; los servicios controlan escrituras y transacciones. SQLAlchemy encapsula persistencia con consultas parametrizadas. No se añade un repositorio que solo duplique cada método del ORM: se puede extraer cuando existan consultas complejas o varios proveedores de persistencia.

2. Autenticación frente a autorización

Autenticación: login verifica el hash scrypt y emite un JWT de 15 minutos. Autorización: cada petición valida firma, emisor, audiencia, expiración, jti revocado, usuario existente, token_version y rol actual en MySQL. Un JWT por sí solo no autoriza una operación.

Rol	CRUD usuarios	Perfil propio	Ghibli
admin	Sí	Sí	Los cinco recursos
films	No	Sí	films
people	No	Sí	people
locations	No	Sí	locations
species	No	Sí	species
vehicles	No	Sí	vehicles

Los roles se asignan explícitamente por un admin. El primer admin se crea desde la CLI en el host autorizado. No existe endpoint de bootstrap público ni contraseña predeterminada. El rol no se confía a una entrada arbitraria del JWT; se consulta su estado actual.

Se aplica autorización antes de cargar cuerpos y de acceder a Ghibli o a su caché. Así, el usuario films no puede acceder a una respuesta people almacenada por un admin. Login usa un hash de verificación de relleno cuando el correo no existe y responde con un mensaje genérico para no revelar cuentas directamente.

3. Integridad y concurrencia

Email se normaliza a minúsculas y tiene restricción UNIQUE; role tiene CHECK en MySQL. La validación de aplicación permite errores útiles y la base protege ante carreras. Una transacción abarca toda la modificación o borrado múltiple. Ante una excepción se ejecuta rollback antes de responder.

La fila única security_lock se bloquea con SELECT FOR UPDATE para cada escritura de usuario. Tras tomar el lock se vuelve a comprobar el actor y se verifica que queda al menos un admin antes del

commit. MySQL usa READ COMMITTED para que una petición que esperó un lock observe el cambio confirmado por la anterior. La prueba concurrente del pipeline elimina simultáneamente dos admins y exige conservar uno.

Decisión: serializar las escrituras de usuarios es simple y protege esta invariante, pero limita su throughput. Es apropiado para un CRUD administrativo de carga moderada. Para cientos de escrituras por segundo conviene separar la protección de admins y diseñar locks más específicos, midiendo primero. SQLite no reproduce locks InnoDB: por eso existe la matriz MySQL en CI.

Borrado físico, no lógico: cumple el alcance y revoca acceso, pero no conserva el historial de usuario. En un sistema con obligaciones de retención habría que definir anonimización, auditoría persistente y borrado lógico antes de ampliar el dominio.

4. Fallos y observabilidad

Ghibli tiene timeout (2 s conexión, 4 s lectura por intento), máximo un reintento de GET, respuestas 502/504 y caché compartida de 60 s. No hay retry de escrituras. No se implementa circuit breaker: en cargas altas se debe añadir para contener una caída prolongada, junto con métricas de latencia y error. Las listas Ghibli utilizan limit; no se promete paginación de un proveedor que no la ofrezca.

El logging registra método, ruta sin query string, estado, duración y un request_id generado por el servidor. Los eventos de alta/cambio/baja identifican actor y destino sin contraseñas, tokens ni cuerpos. No es un sistema de auditoría inmutable. Los errores de dependencias se registran por tipo para evitar filtrar cadenas de conexión.

Liveness valida el proceso. Readiness consulta MySQL y Redis; Ghibli queda fuera para no retirar toda la API por una falla externa. Los balanceadores deben usar readiness, y el monitoreo de Ghibli debe ser independiente. Un healthcheck de Docker detecta enfermedad pero por sí solo no reinicia un contenedor que sigue vivo; un operador o supervisor debe actuar si permanece unhealthy.

5. Escalabilidad y distribución

Los workers Gunicorn no guardan sesiones locales. Redis compartido coordina logout, cuotas y caché. La conexión a MySQL usa pool, pre_ping y reciclado para conexiones cerradas por el servidor. La suma de conexiones posibles crece con workers e instancias: ajustar frente a max_connections y métricas, no aumentar workers a ciegas.

Despliegue incluido: una EC2 x86_64 con Caddy, API, MySQL y Redis en Docker. Es reproducible y sencillo de evaluar, pero hay un único punto de falla y breve interrupción al reemplazar la API. Un volumen Docker no sustituye un backup externo.

Evolución para alta disponibilidad: ALB con certificado ACM -> Auto Scaling con 2+ EC2 en distintas AZ -> RDS MySQL Multi-AZ y Redis compartido con autenticación/TLS. Retirar MySQL/Redis locales, cambiar DATABASE_URL/REDIS_URL, suministrar CA de RDS y exigir TLS a la base. Ajustar ProxyFix al número real de proxies de confianza y restringir los Security Groups. Las migraciones siguen siendo un único job, nunca cada réplica. Esta topología no está aprovisionada por el proyecto.

Las revocaciones son seguridad, no solo caché. Redis usa AOF y noeviction en el despliegue incluido. Si se pierde su volumen, tokens previamente revocados podrían volver a ser válidos hasta expirar; en recuperación de desastre debe rotarse JWT_SECRET_KEY antes de restablecer el servicio. La persistencia no elimina todo riesgo de pérdida. Separar almacenamiento de sesiones y caché al crecer para que la caché no consuma memoria necesaria para revocaciones.

6. CI/CD y cadena de suministro

Pull requests ejecutan lint, tests SQLite/MySQL/Redis, cobertura y auditoría de dependencias. Solo main publica en ECR. GitHub obtiene credenciales efímeras con OIDC, no claves AWS permanentes. SSM ejecuta el script en una instancia específica y el workflow comprueba el estado final remoto.

La aplicación se despliega por digest sha256, no por latest. Las versiones Python están fijadas, pero las imágenes base y acciones usan tags: para un entorno con requisitos de cadena de suministro estrictos, fijar también sus digests/SHAs revisados, añadir escaneo de imagen, SBOM y firma. ECR debe tener escaneo habilitado; el pipeline incluido bloquea vulnerabilidades Python detectadas por pip-audit, pero no implementa una puerta automática de hallazgos ECR.

Las migraciones compatibles preceden al cambio de contenedor. Si falla el arranque, se intenta restaurar la imagen anterior; no se ejecuta downgrade automático de esquema. Usar patrón expand/contract: añadir campos compatibles, desplegar consumidores y solo después retirar campos en otra versión. Si la migración falla, se detiene el despliegue; DDL MySQL puede tener efectos parciales y requiere inspección.

7. Alcance y ampliaciones

No incluye frontend, MFA, recuperación de contraseña, registro público, refresh tokens, circuit breaker, WAF, infraestructura Terraform, ALB ni carga de imágenes. Son ampliaciones explícitas, no funcionalidades simuladas. La configuración, código y pruebas son una base operable que debe validarse en staging con la infraestructura y carga real antes de pasar a producción.

Despliegue paso a paso en AWS EC2

1. Preparar la cuenta y elegir el entorno

Usa una cuenta AWS con permisos para EC2, IAM, ECR, Systems Manager y, si eliges backups externos, S3. Selecciona una región única para ECR/EC2. Necesitas un dominio o subdominio propio con registro A a la IP de la instancia para HTTPS automático de Caddy.

Este procedimiento despliega una sola EC2 con MySQL y Redis en contenedores. Para cargas críticas consulta la evolución a RDS y múltiples AZ en ARQUITECTURA.md. No es alta disponibilidad ni despliegue sin interrupción. AWS cobra por recursos usados; define presupuesto y alertas en tu cuenta antes de crearlos.

2. Crear ECR y roles IAM

Crea un repositorio privado ECR llamado `ghibli-users`, con tags inmutables y escaneo al subir. Desde una terminal AWS autenticada (sustituye la región):

```
aws ecr create-repository --repository-name ghibli-users \
  --image-tag-mutability IMMUTABLE \
  --image-scanning-configuration scanOnPush=true \
  --region us-east-1
```

Crea un rol EC2 con confianza al servicio ec2.amazonaws.com. Asocia AmazonSSMManagedInstanceCore y una política basada en deploy/ec2-ecr-policy.json, sustituyendo REGION y ACCOUNT_ID. Así la instancia puede administrarse por SSM y descargar solo el repositorio permitido. No copies claves AWS al servidor.

En IAM registra el proveedor OIDC `https://token.actions.githubusercontent.com`, audiencia `sts.amazonaws.com`. Crea otro rol para GitHub basado en deploy/github-oidc-trust.json y adjunta deploy/github-deploy-policy.json. Sustituye ACCOUNT_ID, REGION, ORG, REPO e INSTANCE_ID. El claim sub debe coincidir exactamente con tu repositorio y el environment production. Algunas configuraciones recientes de GitHub incluyen IDs inmutables en el subject: confirma el formato real en la documentación de OIDC y adapta la política; no uses un comodín para resolverlo.

La política limita ECR al repositorio y SSM a una instancia. El permiso SendCommand es sensible porque ejecuta código como root en esa EC2: restringe quién puede modificar main y aprobar production.

3. Crear la instancia EC2

Crea una EC2 Ubuntu Server 24.04 LTS x86_64. Como punto inicial para API+MySQL+Redis usa 2 vCPU y 4 GiB de RAM, sujeto a pruebas de carga. El pipeline construye linux/amd64; no elijas Graviton/ARM sin cambiar el build. Usa EBS gp3 cifrado, por ejemplo 30 GiB iniciales, y planifica crecimiento y snapshots.

Asocia el rol EC2 anterior. Exige IMDSv2. Si solo el host necesita credenciales, mantén el acceso a metadata limitado al host; la API no necesita AWS SDK en este alcance. Utiliza una IP estable y un registro DNS A; no crees un AAAA si no configuraste IPv6.

Security Group de entrada: TCP 80 y 443 desde los clientes esperados (Internet si el API es público). No abras 3306, 6379 ni 8000. Administración por Session Manager: no requiere exponer SSH. Permite salida HTTPS para SSM, ECR, Ghibli y certificados, y salida necesaria para instalación/actualización del sistema. Una subred privada requiere NAT o endpoints adecuados; este ejemplo asume salida a Internet y DNS resuelto.

Comprueba que la instancia aparece Online en Systems Manager. En Ubuntu verifica el agente:

```
sudo systemctl status snap.amazon-ssm-agent.amazon-ssm-agent.service
```

Si la AMI no incluye el agente, instálalo siguiendo la documentación AWS. No continúes al pipeline hasta que SSM funcione.

4. Instalar Docker y AWS CLI en EC2

Abre Session Manager. Los siguientes comandos asumen una instancia limpia Ubuntu 24.04, sin instalaciones previas de Docker. Usa el repositorio oficial:

```
sudo apt-get update
sudo apt-get install -y ca-certificates curl unzip python3
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg \
  -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc
. /etc/os-release
printf 'deb [arch=%s signed-by=/etc/apt/keyrings/docker.asc] https://download.docker.com/linux/ubuntu
%s stable\n' \
  "$(dpkg --print-architecture)" "$VERSION_CODENAME" | \
  sudo tee /etc/apt/sources.list.d/docker.list >/dev/null
sudo apt-get update
sudo apt-get install -y docker-ce docker-ce-cli containerd.io \
  docker-buildx-plugin docker-compose-plugin
sudo systemctl enable --now docker
sudo docker compose version
```

Instala AWS CLI v2 con el instalador oficial para x86_64; verifica su firma según la guía de AWS antes de ejecutar la instalación en tu entorno:

```
curl -fLo /tmp/awscliv2.zip \
  https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip
unzip -q /tmp/awscliv2.zip -d /tmp/awscli-install
sudo /tmp/awscli-install/aws/install
aws --version
aws sts get-caller-identity
```

La identidad debe corresponder al rol de EC2. No ejecutes aws configure con claves permanentes. Verifica Docker Compose con soporte para `--wait` y `--wait-timeout`.

5. Copiar archivos operativos y generar secretos

Transfiere el ZIP del proyecto mediante un canal administrativo de confianza. Puedes subirlo a un bucket S3 privado temporal de despliegue, dar al rol EC2 permiso GetObject solo sobre ese objeto y descargarlo desde Session Manager. Retira ese permiso al terminar. El archivo no debe contener .env ni claves. Descomprímelo en /tmp y entra en ghibli-users-api.

Desde ese directorio, ejecuta:

```
sudo install -d -m 700 /opt/ghibli
sudo install -m 600 deploy/compose.prod.yaml /opt/ghibli/compose.prod.yaml
sudo install -m 644 deploy/Caddyfile /opt/ghibli/Caddyfile
sudo install -m 700 deploy/deploy.sh /opt/ghibli/deploy.sh
sudo install -m 700 deploy/backup.sh /opt/ghibli/backup.sh
python3 scripts/init_env.py
sudo install -m 600 .env /opt/ghibli/.env
rm .env
sudo nano /opt/ghibli/.env
```

En /opt/ghibli/.env conserva los secretos aleatorios y añade/ajusta:

```
DOMAIN=api.tudominio.com
TRUST_PROXY=true
ENABLE_DOCS=false
WEB_CONCURRENCY=2
```

DATABASE_URL debe seguir apuntando al host mysql y REDIS_URL a redis dentro de Docker. No escribas valores entre comillas innecesariamente ni uses contraseñas con caracteres URL sin codificarlos; el generador usa hexadecimal para evitar ese problema. `.env` contiene secretos legibles por root y Docker: protege acceso al host y al grupo docker. Para una operación más amplia, carga secretos desde Secrets Manager durante bootstrap; no está automatizado en esta versión.

Caddyfile usa el dominio del .env a través de Compose. El dominio debe resolver correctamente y los puertos 80/443 deben ser alcanzables para emitir el certificado. Guarda copia segura de los secretos para recuperación.

6. Configurar GitHub y ejecutar el primer despliegue

Crea un repositorio y sube el contenido del proyecto con `.github` incluido. El ZIP no contiene un repositorio remoto preconfigurado. Configura protección de main y el environment **production**, permitiendo desplegar solo main. Si tu plan lo permite, añade revisores requeridos.

En Settings -> Environments -> production -> Environment variables configura:

Variable	Ejemplo / contenido
AWS_REGION	us-east-1
AWS_DEPLOY_ROLE_ARN	ARN del rol OIDC de GitHub
ECR_REPOSITORY	ghibli-users
EC2_INSTANCE_ID	ID real de la instancia i-...

No son claves secretas. La contraseña MySQL y JWT_SECRET_KEY permanecen en EC2. Nunca se requieren AWS_ACCESS_KEY_ID ni AWS_SECRET_ACCESS_KEY en GitHub.

Haz push a main. El workflow prueba SQLite/MySQL, aplica migraciones en tests, valida Redis real, lint y pip-audit. Después publica la imagen en ECR y llama por SSM al script preparado. El script hace pull por digest, inicia dependencias, migra una vez, arranca API, verifica readiness y arranca Caddy. GitHub espera el estado final de SSM: enviar un comando no equivale a despliegue correcto.

Las políticas ejemplo y archivos de /opt/ghibli deben prepararse antes del primer push de despliegue. Cambios futuros en deploy.sh, compose.prod.yaml o Caddyfile requieren copiarlos al host mediante el mismo procedimiento administrativo; el pipeline actual actualiza la imagen, no reemplaza automáticamente esos archivos. Prueba dichos cambios en staging primero.

7. Crear el primer administrador y validar

Desde Session Manager, usa una shell root:

```
sudo -i
cd /opt/ghibli
export APP_IMAGE="$(cat current-image)"
docker compose -p ghibli --env-file .env -f compose.prod.yaml \
  exec api flask --app wsgi:app create-admin
docker compose -p ghibli --env-file .env -f compose.prod.yaml ps
curl --fail http://127.0.0.1:8000/health/ready
curl --fail https://api.tudominio.com/health/ready
```

Login y operaciones deben probarse con HTTPS desde un cliente externo. Comprueba: sin token -> 401; admin crea/lista -> 201/200; films consulta films -> 200 y people -> 403; logout seguido de petición -> 401; duplicado -> 409; último admin no se puede eliminar -> 409.

Swagger está deshabilitado en este ejemplo productivo. Para habilitarlo conscientemente, cambia ENABLE_DOCS=true y recrea api con el mismo digest; las operaciones siguen exigiendo JWT. El documento OpenAPI puede seguir siendo accesible sin credenciales: no contiene datos ni secretos. Para documentación estrictamente privada, bloquea /docs y /openapi.json en el proxy.

Si necesitas reiniciar manualmente después de un reboot o mantenimiento, conserva el nombre de proyecto `ghibli` y exporta APP_IMAGE desde current-image. Docker usa restart unless-stopped para los servicios persistentes. `migrate` es un job manual de despliegue, no un servicio permanente.

8. Rollback y migraciones

El script conserva current-image y previous-image. Si la nueva API no llega a healthy, intenta restaurar la imagen anterior. El rollback no revierte SQL. Si no existe una imagen previa (primer despliegue), se informa el fallo para corregirlo.

Rollback manual desde /opt/ghibli como root (sustituye región):

```
bash /opt/ghibli/deploy.sh "$(cat previous-image)" us-east-1
```

Solo aplica si el esquema actual sigue siendo compatible con la imagen anterior. Nunca ejecutes downgrade sin revisar SQL y tener backup comprobado. Para cambios incompatibles usa despliegues por etapas expand/contract y, si procede, ventana de mantenimiento. El primer esquema crea users y security_lock; las revisiones futuras se generan con `flask --app wsgi:app db migrate` en desarrollo y se revisan antes de incluirlas en Git.

9. Backups S3 y restauración

Crea un bucket privado de backups en tu cuenta. Activa Block Public Access, Object Ownership bucket-owner-enforced, cifrado SSE-S3 (o KMS si agregas permisos), versionado y lifecycle según tu retención. Adjunta al rol EC2 deploy/s3-backups-policy.json sustituyendo BACKUP_BUCKET. El script solo necesita PutObject sobre mysql/*; el operador de restauración debe tener GetObject por separado.

Programa diariamente backup.sh con cron de root, usando tu nombre real de bucket y con zona horaria del servidor conocida:

```
0 3 * * * /opt/ghibli/backup.sh mi-bucket-backups >> /var/log/ghibli-backup.log 2>&1
```

El backup usa mysqldump transaccional para InnoDB, comprime, sube a S3 y elimina el temporal. Comparte un lock con deploy.sh para evitar DDL concurrente. Una programación diaria implica potencialmente hasta un día de datos perdidos; el RTO depende de cuánto tarde la restauración. Define RPO/RTO reales y alerta si un backup no aparece. Crea el archivo de log con permiso 600 y configura logrotate.

Prueba una restauración periódica en una base aislada. Descarga un backup autorizado; con los servicios en mantenimiento y una base vacía de recuperación, puedes importar así (desde /opt/ghibli, root):

```
export APP_IMAGE="$(cat current-image)"
gunzip -c /ruta/backup.sql.gz | \
  docker compose -p ghibli --env-file .env -f compose.prod.yaml \
  exec -T mysql sh -c 'MYSQL_PWD="$MYSQL_PASSWORD" mysql -u "$MYSQL_USER" "$MYSQL_DATABASE"'
```

Ese comando reemplaza tablas del destino con el dump: úsalo solo en el destino de recuperación elegido, no sobre producción activa. Comprueba recuentos, usuarios, migración alembic_version y login. El dump

incluye hashes de contraseñas y debe tratarse como dato confidencial. Conserva también secretos, revisión de código y configuración. Snapshots EBS ayudan, pero deben complementarse con restauraciones probadas.

Redis conserva revocaciones en volumen AOF. Si se pierde ese volumen, rota JWT_SECRET_KEY y recrea API antes de reabrir tráfico, para invalidar todos los tokens antiguos. La pérdida de caché Ghibli es tolerable; la pérdida de revocaciones requiere esta acción.

10. Monitoreo y mantenimiento

Revisa logs de API/Caddy, estado de contenedores, espacio EBS, memoria, CPU, errores 5xx, latencia y conexiones MySQL. Usa CloudWatch Agent si requieres métricas de memoria/disco y centralización de logs; su instalación/configuración no está incluida. Configura alarmas y notificación a un canal operativo de tu equipo.

Comandos útiles, con APP_IMAGE exportado como en el paso 7:

```
docker compose -p ghibli --env-file .env -f compose.prod.yaml logs --tail=100 api
docker compose -p ghibli --env-file .env -f compose.prod.yaml logs --tail=100 caddy
docker system df
df -h
```

Nunca registres Authorization, contraseñas ni cuerpos completos. No uses docker system prune --volumes en un host con datos. Habilita actualizaciones de seguridad del sistema y revisa cambios de dependencias en staging. Antes de cambiar versiones mayores de MySQL, planifica y prueba la migración de datos.

11. S3 para futuras imágenes (opcional, no implementado)

El API actual no guarda archivos y no requiere bucket de imágenes. Si agregas avatares: usa un bucket privado distinto del de backups, Block Public Access, cifrado y rol IAM sin claves estáticas. Guarda en MySQL solo la clave del objeto. Genera claves nuevas por usuario y versión, nunca rutas arbitrarias recibidas del cliente.

Añade validación de pertenencia, límite de tamaño, detección real del formato y decodificación/recodificación de la imagen antes de publicarla. Para subida directa usa POST prefirmado con límite content-length-range y un prefijo por usuario; posteriormente valida el objeto desde backend antes de asociarlo al perfil. Un Content-Type declarado por el cliente no basta.

Configura AWS_REGION y S3_IMAGES_BUCKET, permisos PutObject/GetObject/DeleteObject limitados al prefijo avatars/* y URLs prefirmadas de lectura con expiración corta. Si el SDK corre en el contenedor, planifica acceso seguro al rol de EC2/metadata o migra a ECS con task roles; no incrustes credenciales en Docker. Esta ampliación necesita endpoints, migración y pruebas adicionales, no se activa con solo variables de entorno.

12. Fallas frecuentes

• MySQL no está healthy: revisar contraseña, disco y logs. No borrar el volumen para resolver un problema en producción.

• API responde 503: verificar MySQL, Redis, migraciones y conectividad. La política falla cerrado si no se puede validar revocación.

• SSM no ve la instancia: verificar agente, instance profile, salida HTTPS y DNS.

• OIDC AccessDenied: verificar audience, sub exacto y environment production, incluyendo posible formato de IDs inmutables.

• Caddy no consigue certificado: revisar A/AAAA, puertos 80/443 y DNS.

• Ghibli 502/504: comprobar disponibilidad del proveedor y GHIBLI_BASE_URL; no quitar autorización para probarlo.

• ECR rechaza un tag existente: los tags son inmutables. Un reintento de build del mismo commit puede fallar al publicar; usa un nuevo commit o despliega el digest ya existente de forma controlada.

• Pipeline terminó sin confirmar SSM: inspeccionar Run Command antes de repetir, pues el comando remoto puede seguir ejecutándose.

Validación del entregable

Fecha: 26 de octubre de 2026. Entorno disponible: Python 3.12.14 sobre Linux.

Ejecutado y aprobado

• Pytest: 45 pruebas aprobadas y 2 omitidas con motivo explícito.

• Cobertura de aplicación: 97,15% (409 de 421 sentencias cubiertas); umbral configurado: 80%.

• Migración inicial Alembic aplicada en bases SQLite temporales de las pruebas.

• CRUD, normalización, duplicados, transacciones atómicas, último admin, expiración, logout, invalidación por rol/contraseña y matriz de los seis roles.

• Ghibli: transporte HTTP simulado; validación de caché, timeout, fallo de conexión, HTTP adverso, forma de respuesta y límites.

• Ruff: sin errores. Compilación Python: correcta. pip check: sin incompatibilidades declaradas.

• pip-audit: sin vulnerabilidades conocidas detectadas para requirements.txt en la consulta realizada. No garantiza ausencia de vulnerabilidades futuras ni audita el sistema operativo de la imagen.

• YAML/JSON: lectura sintáctica correcta. Scripts Bash: bash -n sin errores.

• Swagger / OpenAPI: endpoint generado correctamente y copia estática incluida en docs/openapi.json.

Pendiente en infraestructura de destino

No hay Docker, MySQL ni Redis reales disponibles en el entorno de elaboración. Por ello no se ejecutaron aquí la construcción de imagen, docker compose up, el healthcheck de contenedor ni las dos pruebas específicas de Redis real y concurrencia InnoDB. La matriz GitHub Actions las ejecutará al subir el proyecto, pero el workflow aún no se ha ejecutado en tu repositorio.

No se han creado recursos AWS, iniciado EC2, publicado en ECR, configurado un dominio ni probado SSM/OIDC contra tu cuenta. Las políticas IAM requieren sustituir los marcadores por tus datos. Tampoco se ha comprobado la disponibilidad efectiva de los endpoints Ghibli mediante una llamada productiva autenticada del proyecto; las pruebas externas usan mocks para ser deterministas.

Antes de producción: ejecutar el pipeline completo, validar despliegue y TLS en staging, realizar las pruebas manuales de roles, probar backup/restauración y medir consumo de recursos/carga. No confundir cobertura de líneas con seguridad certificada o preparación operacional completa.

Referencias consultadas

Consulta: 26 de octubre de 2026. Las decisiones y el código son específicos de este proyecto; estas fuentes sustentan las herramientas y procedimientos, no certifican el despliegue.

• Flask / Gunicorn: https://flask.palletsprojects.com/en/stable/deploying/gunicorn/

• API Ghibli configurada (servicio comunitario): https://ghibliapi.vercel.app/

• GitHub Actions, OIDC con AWS: https://docs.github.com/en/actions/how-tos/secure-your-work/security-harden-deployments/oidc-in-aws

• Systems Manager Run Command: https://docs.aws.amazon.com/systems-manager/latest/userguide/run-command.html

• Estado del agente SSM: https://docs.aws.amazon.com/systems-manager/latest/userguide/ssm-agent-status-and-restart.html

• Docker Engine en Ubuntu: https://docs.docker.com/engine/install/ubuntu/

• Instalación AWS CLI: https://docs.aws.amazon.com/cli/latest/userguide/getting-started-install.html

• Seguridad de S3: https://docs.aws.amazon.com/AmazonS3/latest/userguide/security-best-practices.html